新排课系统2024_09_20

遗传算法2024_09_23

遗传算法概念2024_09_23

遗传算法是一种模拟自然选择过程的优化方法，适用于求解复杂的组合优化问题，如排课问题。在排课问题中，需要在多个时间段和教室之间安排一系列课程，并尽量满足一系列约束条件（如避免冲突、教师可用性、教室容量等）。以下是遗传算法在排课问题中的基本工作流程：

1.初始种群生成 (Initialization)

- **个体表示 (Chromosome Representation)**: 在排课问题中，每个个体（染色体）表示一种排课方案。其中染色体上的每个基因元素表示某门课程在某个时间段和教室的安排。
- **种群初始化 (Population Initialization)**: 生成一定数量的随机排课方案，作为初始种群。

2.适应度评估 (Fitness Evaluation)

- 为每个个体计算一个适应度值（Fitness Value），表示该排课方案的优劣。适应度值的计算根据以下几个方面：
  - 是否存在时间冲突（如同一时间段安排了多门课程在同一个教室）。
  - 教师的排课均衡（如教师是否安排多次或未安排课程）。
  - 课程优先级（如优先级高的课程是否排课）。
  - 禁止排课时间（如课程不可以安排在某一时间段）。

3.选择 (Selection)

- 根据适应度值选择父代个体，优良个体被选择的概率更高。常见的选择方法包括轮盘赌选择 (Roulette Wheel Selection)、锦标赛选择 (Tournament Selection) 等。这里以适应度排名只选取适应度最高部分几位作为父母

4.交叉 (Crossover)

- 从选择的父代个体中随机挑选两个进行交叉操作，生成新的后代个体（即新的排课方案）。常用的交叉方法有：
  - 单点交叉 (Single-Point Crossover)：在染色体的某一位置切分，交换两部分。
  - 多点交叉 (Multi-Point Crossover)：在多个位置切分，并交换切分的片段。

5. 变异 (Mutation)

- 对后代个体进行小概率的随机变异操作，以增加种群的多样性。对于排课问题，变异表现为随机调整某门课程的时间和教室安排。

6. 新一代种群形成 (New Population)

- 由选择、交叉和变异操作生成的新个体和适应度最高的父代个体构成新一代种群。（为了维持种群质量，将前一代种群中最优的个体（精英保留）直接保留到新一代。）

7. 终止条件 (Termination Condition)

- 算法通常在满足以下条件之一时终止：

    - 达到了最大迭代次数。
    - 适应度值没有显著提升（收敛）。
    - 找到了满足所有约束条件的最优排课方案（即适应度值达到设置的阈值）。

8. 解的提取 (Solution Extraction)

- 从最终种群中选择适应度值最高的个体，作为最终的排课方案。

优点

- **灵活性**：可以处理多种复杂的约束条件。
- **全局优化能力**：避免陷入局部最优解。
- **适应性强**：可以适用于不同规模和复杂度的排课问题。

缺点

- **计算成本高**：对于大型问题，算法收敛速度较慢。
- **参数调节复杂**：种群大小、交叉概率、变异概率等参数的选择对算法性能影响较大。

整体评价

遗传算法在排课问题中的应用具有较高的有效性和灵活性，实际实现中需要根据具体问题对适应度函数和参数进行调优。

遗传算法具体实现2024_09_24

下面结合代码细节来介绍遗传算法的各个步骤。

1. 课程、课程表和学生类的定义

- `Course` **类**：定义课程的基本属性，包括课程名称、教师名称、持续时间、优先级和不可用时间段。
- `Student` **类**：表示学生，包含学生姓名和选课列表，以及已上课的时间段。上课时间段根据排课安排自动生成
- `Schedule` **类**：维护一个课程表，包含课程、时间段和教室的信息。初始化时，所有时间段和教室组合都是空的。

2. 适应度评估 (Fitness Function)

```
fitness
```

方法通过以下方式评估一个排课方案的适应度：

- **课程冲突**：同一课程在同一时间安排到多个教室将扣分。
- **教师安排**：教师被合理安排课程的次数，过多或过少都会影响适应度。
- **学生冲突**：学生不能在同一时间上多门课，冲突时会大幅降低适应度。
- **教师不可用时间**：如果课程安排在教师不可用的时间，适应度会降低。

3. 变异 (Mutation)

- `mutate` 方法随机选择一个时间段和教室组合，然后随机更改此时间和教室安排的课程。增加种群基因的多样性，探索更多的可能性。

4. 交叉 (Crossover)

- `crossover` 方法从几个父代排课方案中生成两个子代。它随机选择一个交叉点，然后交换交叉点后的部分课程，生成新的排课方案。这有助于保留良好的排课特性，并组合不同个体的优点。

5. 遗传算法流程

1. **初始化种群**：生成一定数量的随机课程安排作为初始种群。
2. **进化过程**：
    - 计算每个个体的适应度。
    - 保留适应度最高的个体。
    - 随机选择高适应度个体进行交叉和变异，生成下一代。
    - 重复直到找到满足适应度阈值的最佳安排。

遗传算法排课具体代码（python）2024_09_24

```python
'''
Author: xudawu
Date: 2024-09-18 16:39:56
LastEditors: xudawu
LastEditTime: 2024-09-25 11:49:06
'''
import random

class Course:
    def __init__(self, name_str, teacher_str, duration_float, priority_float=1, unavailable_timeslots_list=[]):
        self.name_str = name_str
        self.teacher_str = teacher_str
        self.duration_float = duration_float
        self.priority_float = priority_float
        # 不可用的时间列表
        self.unavailable_timeslots_list = unavailable_timeslots_list if unavailable_timeslots_list else []

class Student:
    def __init__(self, name_str):
        self.name_str = name_str
        self.enrolled_courses_list = []
        # 记录学生的上课时间段
        self.time_slots_list = []

    def enroll(self, course):
        self.enrolled_courses_list.append(course)

class Schedule:
    def __init__(self, course_list, timeslots_list, room_list, student_list):
        self.course_list = course_list
        self.timeslots_list = timeslots_list
        self.room_list = room_list
        self.student_list = student_list
        # 时段和教室的组合作为键，课程作为值,课程初始化为None
        self.schedule_dict = {}
        for cur_timeslot_str in self.timeslots_list:
            for cur_room_str in self.room_list:
                self.schedule_dict[(cur_timeslot_str, cur_room_str)] = None

    def fitness(self):
        fitness_float = 0
```

```python
        course_time_dict = {}
        teacher_assigned_dict = {}

        # 初始化学生冲突时间表为空列表
        for CurStudent in self.student_list:
            CurStudent.time_slots_list = []

        # 初始化冲突信息的列表
        self.conflict_info_list = []

        # 每个教师被安排的次数初始化为0
        for CurCourse in self.course_list:
            teacher_assigned_dict[CurCourse.teacher_str] = 0

        for (cur_timeslot_str, cur_room_str), CurCourse in self.schedule_dict.items():
            # 如果当前时段有课程安排,则进行判断
            if CurCourse is not None:
                # 同一课程同一时间不能安排在多个教室,同一时间同一课程名代表有冲突
                # 在安排中则代表有冲突
                if (cur_timeslot_str, CurCourse.name_str) in course_time_dict:
                    fitness_float -= 3
                else:
                    # 没有在安排表中，记录此次安排
                    course_time_dict[(cur_timeslot_str, CurCourse.name_str)] = cur_room_str
                    # 安排此课,适应度分值加上优先级分值
                    fitness_float += CurCourse.priority_float
                    # 统计教师被安排的次数
                    teacher_assigned_dict[CurCourse.teacher_str] += 1

                    # 学生不能同时上两门及以上课程,进行冲突检测
                    for CurStudent in self.student_list:
                        if CurCourse in CurStudent.enrolled_courses_list:
                            if cur_timeslot_str in CurStudent.time_slots_list:
                                # 有冲突大幅降低适应度
                                fitness_float -= 3
                                # 记录冲突信息
                                self.conflict_info_list.append((CurStudent.name_str, CurCourse.name_str, cur_timeslot_str))

                            # 更新学生的时间段表,增加此上课时间到学生的上课时间表中
                            else:
                                CurStudent.time_slots_list.append(cur_timeslot_str)

                # 如果此门课程的安排时间在教师不可以上课的时间列表中，适应度大幅降低
                if cur_timeslot_str in CurCourse.unavailable_timeslots_list:
                    fitness_float -= 3

        # 教师排课均衡性评分
        for course_teacher_name_str, teacher_assigned_count in teacher_assigned_dict.items():
            # 教师没有被分配到课程
```

```python
            if teacher_assigned_count == 0:
                fitness_float -= 5
            # 教师被分配了大于等于3次课程
            elif teacher_assigned_count >= 3:
                fitness_float -= 2
            # 教师排课次数均衡
            else:
                fitness_float += 1

        # 返回适应度
        return fitness_float

    # 基因变异操作
    def mutate(self):
        # 随机选择一个时间段和教室组合
        random_timeslot_room_tuple = random.choice(list(self.schedule_dict.keys()))
        # 随机选择一门课程
        random_new_course_str = random.choice(self.course_list)
        # 替换该位置的课程
        self.schedule_dict[random_timeslot_room_tuple] = random_new_course_str

    # 基因交叉操作
    def crossover(self, other):
        # 随机选择交叉基因的起点
        crossover_index_int = random.randint(0, len(self.schedule_dict) - 1)
        # 初始化两个子代的基因
        child1_schedule_dict = {}
        child2_schedule_dict = {}

        # 迭代所有时间和房间的键值对
        for i, schedule_key_tuple in enumerate(self.schedule_dict.keys()):
            # 子代1交叉基因起点之前的基因来自本个体,子代2的基因来自其他个体
            if i <= crossover_index_int:
                child1_schedule_dict[schedule_key_tuple] = self.schedule_dict[schedule_key_tuple]
                child2_schedule_dict[schedule_key_tuple] = other.schedule_dict[schedule_key_tuple]
            else:
            # 子代1交叉基因起点之后的基因来自其他个体,子代2的基因来自本个体
                child1_schedule_dict[schedule_key_tuple] = other.schedule_dict[schedule_key_tuple]
                child2_schedule_dict[schedule_key_tuple] = self.schedule_dict[schedule_key_tuple]

        # 初始化两个子代类的基因
        Child1Schedule = Schedule(self.course_list, self.timeslots_list, self.room_list, self.student_list)
        Child2Schedule = Schedule(self.course_list, self.timeslots_list, self.room_list, self.student_list)
        # 赋予子代基因
        Child1Schedule.schedule_dict = child1_schedule_dict
        Child2Schedule.schedule_dict = child2_schedule_dict

        # 返回两个新子代基因
        return Child1Schedule, Child2Schedule
```

```python

    def display(self):
        # 存储未安排课程的教师、学生和教室
        unassigned_teachers = set(teacher.teacher_str for teacher in self.course_list)
        unassigned_rooms = set(self.room_list)
        unassigned_students = set(student.name_str for student in self.student_list)

        for (cur_timeslot_str, cur_room_str), CurCourse in self.schedule_dict.items():
            # 如果课程为非空,则打印课程信息
            if CurCourse is not None:
                # 初始化本门课的选课学生列表
                enrolled_course_student_name_list = []
                # 遍历学生列表
                for CurStudent in self.student_list:
                    # 如果本门课在当前学生选课列表中,则将学生姓名加入选课学生列表
                    if CurCourse in CurStudent.enrolled_courses_list:
                        enrolled_course_student_name_list.append(CurStudent.name_str)
                        # 从未安排列表中移除学生
                        unassigned_students.discard(CurStudent.name_str)
                enrolled_course_student_str = ", ".join(enrolled_course_student_name_list) if enrolled_course_student_name_list else "No student enrolled"

                # 检查教师是否在不可上课时间
                teacher_unavailable = cur_timeslot_str in CurCourse.unavailable_timeslots_list
                availability_str = " | Teacher unavailable during this time" if teacher_unavailable else ""


                # 初始化冲突信息
                conflict_str = ""
                for conflict_info_tuple in self.conflict_info_list:
                    if conflict_info_tuple[1] == CurCourse.name_str and conflict_info_tuple[2] == cur_timeslot_str:
                        conflict_str = f" | Conflict: {conflict_info_tuple[0]}"

                print(f"{cur_timeslot_str} in {cur_room_str}: {CurCourse.name_str} by {CurCourse.teacher_str} (Priority: {CurCourse.priority_float}) | Enrolled: {enrolled_course_student_str} {conflict_str}{availability_str}")

                # 从未安排列表中移除教师和教室
                unassigned_teachers.discard(CurCourse.teacher_str)
                unassigned_rooms.discard(cur_room_str)
            else:
                print(f"{cur_timeslot_str} in {cur_room_str}: Free")

        # 打印未安排的教师、教室和学生
        if unassigned_teachers:
            print("Unassigned Teachers: " + ", ".join(unassigned_teachers))
        else:
```

```python
            print("All teachers assigned.")

        if unassigned_rooms:
            print("Unassigned Rooms: " + ", ".join(unassigned_rooms))
        else:
            print("All rooms assigned.")

        if unassigned_students:
            print("Unassigned Students: " + ", ".join(unassigned_students))
        else:
            print("All students enrolled.")


# 初始化种群
def initialize_population(course_list, timeslots_list, room_list, student_list, population_size=100):
    # 种群列表初始化
    population_list = []
    for _ in range(population_size):
        # 实例化一个个体
        SingleSchedule = Schedule(course_list, timeslots_list, room_list, student_list)
        # 对个体的基因进行随机初始化
        for schedule_key_tuple in SingleSchedule.schedule_dict.keys():
             # 随机一个基因(对时间段和教室的组合随机选择一门课)
            SingleSchedule.schedule_dict[schedule_key_tuple] = random.choice(course_list)

        # 将初始化后的个体加入种群列表
        population_list.append(SingleSchedule)

    # 返回初始化的种群列表
    return population_list

# 交叉和变异获得下一代种群
def genetic_evolution(population_list,retention_number_int):

    # 计算适应度,并对种群列表进行倒序排序
    population_list.sort(key=lambda x: x.fitness(), reverse=True)

    # 初始化下一代个体列表
    next_generation_list = []
    # 保留最优的部分个体
    for i in range(retention_number_int):
        next_generation_list.append(population_list[i])

    # 选择和交叉,直到达到种群大小
    while len(next_generation_list) < len(population_list):
        # 随机在适应度最高的部分中选择父母
        parent1 = random.choice(population_list[:10])
        parent2 = random.choice(population_list[:10])
        # 对父母基因进行交叉,得到两个子代
        child1, child2 = parent1.crossover(parent2)
        # 将两个子代加入下一代个体列表
        next_generation_list.append(child1)
        next_generation_list.append(child2)

```

```python
    # 子代和父代有概率变异
    for ChildsChedule in next_generation_list:
        if random.random() < 0.3:
            ChildsChedule.mutate()

    return next_generation_list

if __name__ == "__main__":
    # 示例课程（课程名、任课教师、课时长(单位小时)、优先级(数字越高优先级越高)、不可上课时间）
    course_list = [
        Course(
            name_str="class1", teacher_str='teacher1', duration_float=1,
            priority_float=1, unavailable_timeslots_list=["time2", "time4"]
            ),
        Course("class2", "teacher2", 1, 1, ["time1"]),
        Course("class3", "teacher3", 1, 1),
        Course("class4", "teacher4", 1, 1, ["time3"]),
        Course("class5", "teacher5", 1, 1),
        Course("class6", "teacher6", 1, 1, ["time5", "time6"]),
        Course("class7", "teacher7", 1, 1),
        Course("class8", "teacher8", 1, 1, ["time2"]),
        Course("class9", "teacher9", 1, 1),
        Course("class10", "teacher10", 1, 1, ["time1", "time6"]),
        Course("class11", 'teacher11', 1, 1),
        Course("class12", "teacher12", 1, 1, ["time1"]),
        Course("class13", "teacher13", 1, 1),
        Course("class14", "teacher14", 1, 1, ["time3"]),
        Course("class15", "teacher15", 1, 1),
        Course("class16", "teacher16", 1, 1, ["time5"]),
        Course("class17", "teacher17", 1, 2),
        Course("class18", "teacher18", 1, 2, ["time2"]),
        Course("class19", "teacher19", 1, 3),
        Course("class20", "teacher20", 1, 3, ["time4"]),
    ]

    # 实例时间段和教室
    timeslots_list = ["time1", "time2", "time3", "time4", "time5", "time6"]
    room_list = ["Room1", "Room2", "Room3"]

    # 示例学生
    student_list = [
        Student("student1"),Student("student2"),Student("student3"),
        Student("student4"),Student("student5"),Student("student6"),
        Student("student7"),Student("student8"),Student("student9"),
        Student("student10"),
    ]

    # 学生选课
    student_list[0].enroll(course_list[0])
    student_list[0].enroll(course_list[1])
    student_list[1].enroll(course_list[2])
    student_list[1].enroll(course_list[3])
    student_list[2].enroll(course_list[4])
    student_list[2].enroll(course_list[12])
    student_list[2].enroll(course_list[13])
```

```python
    student_list[2].enroll(course_list[14])
    student_list[3].enroll(course_list[5])
    student_list[3].enroll(course_list[15])
    student_list[3].enroll(course_list[16])
    student_list[4].enroll(course_list[6])
    student_list[4].enroll(course_list[17])
    student_list[4].enroll(course_list[18])
    student_list[4].enroll(course_list[19])
    student_list[5].enroll(course_list[7])
    student_list[6].enroll(course_list[8])
    student_list[7].enroll(course_list[9])
    student_list[8].enroll(course_list[10])
    student_list[9].enroll(course_list[11])

    # 序号2学生多选课程
    student_list[2].enroll(course_list[15])
    student_list[2].enroll(course_list[16])
    student_list[2].enroll(course_list[17])
    student_list[2].enroll(course_list[18])

    # 种群大小和适应度阈值
    population_size_int = 100
    fitness_threshold_float = 50
    # 保留最优的多少个
    retention_number_int=5

    # 获得初始化的种群
    population_list = initialize_population(course_list, timeslots_list, room_list, student_list, population_size_int)

    # 初始化进化次数
    generation_int = 0
    while True:
        # 交叉和变异获得下一代种群
        population_list = genetic_evolution(population_list,retention_number_int)

        # 获取最优个体
        best_schedule = max(population_list, key=lambda x: x.fitness())
        best_fitness_float = best_schedule.fitness()

        # 打印当前代的最优适应度
        generation_int += 1
        print(f"Generation {generation_int}: Best Fitness = {best_fitness_float}")
        best_schedule.display()
        print("-" * 40)

        # 如果达到了适应度要求，退出循环
        if best_fitness_float >= fitness_threshold_float:
            break

    print("Final Best Schedule:")
    best_schedule.display()

```

排课模拟数据2024_09_25

示例课程（课程名、任课教师、课时长(单位小时)、优先级(数字越高优先级越高)、不可上课时间)

```
course_list = [
Course(
       name_str="class1", teacher_str='teacher1', duration_float=1,
       priority_float=1, unavailable_timeslots_list=["time2", "time4"]
       ),
Course("class2", "teacher2", 1, 1, ["time1"]),
Course("class3", "teacher3", 1, 1),
Course("class4", "teacher4", 1, 1, ["time3"]),
Course("class5", "teacher5", 1, 1),
Course("class6", "teacher6", 1, 1, ["time5", "time6"]),
Course("class7", "teacher7", 1, 1),
Course("class8", "teacher8", 1, 1, ["time2"]),
Course("class9", "teacher9", 1, 1),
Course("class10", "teacher10", 1, 1, ["time1", "time6"]),
Course("class11", 'teacher11', 1, 1),
Course("class12", "teacher12", 1, 1, ["time1"]),
Course("class13", "teacher13", 1, 1),
Course("class14", "teacher14", 1, 1, ["time3"]),
Course("class15", "teacher15", 1, 1),
Course("class16", "teacher16", 1, 1, ["time5"]),
Course("class17", "teacher17", 1, 2),
Course("class18", "teacher18", 1, 2, ["time2"]),
Course("class19", "teacher19", 1, 3),
Course("class20", "teacher20", 1, 3, ["time4"]),
    ]
```

示例时间段和教室

```
# 实例时间段和教室
timeslots_list = ["time1", "time2", "time3", "time4", "time5", "time6"]
room_list = ["Room1", "Room2", "Room3"]
```

示例学生

```
# 示例学生
student_list = [
    Student("student1"),Student("student2"),Student("student3"),
    Student("student4"),Student("student5"),Student("student6"),
    Student("student7"),Student("student8"),Student("student9"),
    Student("student10"),
    ]
```

示例学生选课

```
# 学生选课
student_list[0].enroll(course_list[0])
student_list[0].enroll(course_list[1])
student_list[1].enroll(course_list[2])
student_list[1].enroll(course_list[3])
student_list[2].enroll(course_list[4])
student_list[2].enroll(course_list[12])
student_list[2].enroll(course_list[13])
student_list[2].enroll(course_list[14])
student_list[3].enroll(course_list[5])
student_list[3].enroll(course_list[15])
student_list[3].enroll(course_list[16])
student_list[4].enroll(course_list[6])
student_list[4].enroll(course_list[17])
student_list[4].enroll(course_list[18])
student_list[4].enroll(course_list[19])
student_list[5].enroll(course_list[7])
student_list[6].enroll(course_list[8])
student_list[7].enroll(course_list[9])
student_list[8].enroll(course_list[10])
student_list[9].enroll(course_list[11])

# 序号2学生多选课程
student_list[2].enroll(course_list[15])
student_list[2].enroll(course_list[16])
student_list[2].enroll(course_list[17])
student_list[2].enroll(course_list[18])
```

模拟结果

种群大小为100、个体变异单个基因的概率为30%，设置适应度大于等于50代表收敛

适应度增加和减少规则：

- 课程时间冲突：同一课程在同一时间在多个教室授课，减少3分适应度
- 学生时间冲突：同一学生在同一时间上多门课程，减少3分适应度
- 不可上课时间：课程安排上课时间为教师不可上课时间，减少3分适应度
- 排课是否均衡：教师未被安排授课，减少5分适应度；教师被安排授课次数大于等于3次，减少2分适应度；排课次数为1-2次，增加1分适应度
- 课程的优先级：安排课程时，适应度增加课程优先级的分数（如课程优先级为3，则增加3分适应度）

以下为排课模拟结果（前3代的进化结果）

```
Generation 1: Best Fitness = 3
time1 in Room1: class19 by teacher19 (Priority: 3) | Enrolled: student3, student5
time1 in Room2: class1 by teacher1 (Priority: 1) | Enrolled: student1
time1 in Room3: class4 by teacher4 (Priority: 1) | Enrolled: student2
time2 in Room1: class13 by teacher13 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class13 by teacher13 (Priority: 1) | Enrolled: student3
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class15 by teacher15 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class5 by teacher5 (Priority: 1) | Enrolled: student3
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class14 by teacher14 (Priority: 1) | Enrolled: student3 | Conflict: student3
Unassigned Teachers: teacher8, teacher3, teacher16, teacher18, teacher17, teacher7
All rooms assigned.
Unassigned Students: student6
----------------------------------------
Generation 2: Best Fitness = 14
time1 in Room1: class7 by teacher7 (Priority: 1) | Enrolled: student5
time1 in Room2: class3 by teacher3 (Priority: 1) | Enrolled: student2
time1 in Room3: class17 by teacher17 (Priority: 2) | Enrolled: student3, student4
time2 in Room1: class13 by teacher13 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class13 by teacher13 (Priority: 1) | Enrolled: student3
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class15 by teacher15 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class5 by teacher5 (Priority: 1) | Enrolled: student3
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class14 by teacher14 (Priority: 1) | Enrolled: student3 | Conflict: student3
Unassigned Teachers: teacher8, teacher16, teacher18, teacher19
All rooms assigned.
Unassigned Students: student6
----------------------------------------
----------------------------------------
Generation 3: Best Fitness = 17
time1 in Room1: class7 by teacher7 (Priority: 1) | Enrolled: student5
time1 in Room2: class3 by teacher3 (Priority: 1) | Enrolled: student2
time1 in Room3: class17 by teacher17 (Priority: 2) | Enrolled: student3, student4
time2 in Room1: class13 by teacher13 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class13 by teacher13 (Priority: 1) | Enrolled: student3
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class15 by teacher15 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class8 by teacher8 (Priority: 1) | Enrolled: student6
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class14 by teacher14 (Priority: 1) | Enrolled: student3
Unassigned Teachers: teacher16, teacher18, teacher5, teacher19
All rooms assigned.
All students enrolled.
----------------------------------------
```

第1代种群最高适应度为3，最高适应度个体基因中有6门课程未进行排课，其中teacher17、teacher18的课程优先级为2，teacher1在time1的不可上课时间进行了排课；在time6中student3出现上课时间冲突，共有1名学生没有排课安排

第2代种群最高适应度为14，最高适应度个体基因中有4门课未进行排课，优先级2的teacher17安排了排课，但优先级为3的teacher19未安排排课，没有教师在不可上课时间安排了排课；在time6出现学生上课时间冲突，共有1名学生没有排课安排

第3代种群最高适应度为17，最高适应度个体基因中有4门课未进行排课，其中teacher18、teacher19的课程优先级为2和3，没有教师在不可上课时间安排了排课；所有学生上课时间无冲突，所有学生安排了排课

.....（省略部分、以下只记录适应度变化的节点）.....

第8代种群最高适应度为28，进化到第24代种群最高适应度为29，到第251最高适应度为32，到2460代最高适应度依然为32

以下为第2460代适应度为32的个体基因（排课安排），因设置的资源配置，必定有两门课无法安排排课

```
----------------------------------------
Generation 2460: Best Fitness = 32
time1 in Room1: class7 by teacher7 (Priority: 1) | Enrolled: student5
time1 in Room2: class3 by teacher3 (Priority: 1) | Enrolled: student2
time1 in Room3: class17 by teacher17 (Priority: 2) | Enrolled: student3, student4
time2 in Room1: class15 by teacher15 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class19 by teacher19 (Priority: 3) | Enrolled: student3, student5
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class5 by teacher5 (Priority: 1) | Enrolled: student3
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class13 by teacher13 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class8 by teacher8 (Priority: 1) | Enrolled: student6
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class18 by teacher18 (Priority: 2) | Enrolled: student3, student5
Unassigned Teachers: teacher16, teacher14
All rooms assigned.
All students enrolled.
----------------------------------------
```

第2460代种群最高适应度为32，优先级大于1的课程全部安排排课，未进行排课的teacher16和teacher14的课程优先级为1（默认最低为1）；没有教师上课时间冲突；没有学生上课时间冲突；没有教师排课安排在不可上课时间；从第1代种群到第2460种群，用时9秒04

结论2024_09_25

实验结果来看，使用遗传算法进行排课可以满足优化排课和快速排课的要求，遗传算法优点在于可以根据需要自由设计适应度计算方法，但需要注意遗传进化算法对适应度计算参数敏感，需要根据需要详细设计适应度计算函数；

具体代码JavaScript版本（基于ChatGPT4o转换，可能有错）

```javascript
// Author: xudawu
// Date: 2024-09-18 16:39:56
// LastEditors: xudawu
// LastEditTime: 2024-09-25 11:49:06

class Course {
    constructor(name, teacher, duration, priority = 1, unavailableTimeslots = []) {
        this.name = name; // 课程名称
        this.teacher = teacher; // 教师名称
        this.duration = duration; // 课程时长
        this.priority = priority; // 课程优先级
        this.unavailableTimeslots = unavailableTimeslots.length > 0 ? unavailableTimeslots : []; // 不可用时间列表
    }
}

class Student {
    constructor(name) {
        this.name = name; // 学生名称
        this.enrolledCourses = []; // 已选课程列表
        this.timeSlots = []; // 学生的上课时间段记录
    }

    enroll(course) {
        this.enrolledCourses.push(course); // 学生选课
    }
}

class Schedule {
    constructor(courseList, timeslotsList, roomList, studentList) {
        this.courseList = courseList; // 课程列表
        this.timeslotsList = timeslotsList; // 时间段列表
        this.roomList = roomList; // 教室列表
        this.studentList = studentList; // 学生列表
        this.scheduleDict = {}; // 存储时段与教室组合的课程安排

        // 初始化安排字典
        for (let timeslot of this.timeslotsList) {
            for (let room of this.roomList) {
                this.scheduleDict[`${timeslot}-${room}`] = null; // 课程初始化为 null
            }
        }
    }

    fitness() {
        let fitnessScore = 0; // 适应度分数
        let courseTimeDict = {}; // 记录课程时间安排
        let teacherAssignedCount = {}; // 记录每位教师被安排的次数

        // 初始化学生的时间冲突列表
        for (let student of this.studentList) {
            student.timeSlots = [];
```

```
        }

        // 初始化冲突信息列表
        this.conflictInfoList = [];

        // 初始化教师安排计数
        for (let course of this.courseList) {
            teacherAssignedCount[course.teacher] = 0;
        }

        // 遍历安排字典
        for (let key in this.scheduleDict) {
            let [timeslot, room] = key.split('-');
            let course = this.scheduleDict[key];

            if (course) {
                // 检测同一课程同一时间在多个教室的冲突
                if (courseTimeDict[`${timeslot}-${course.name}`]) {
                    fitnessScore -= 3; // 发生冲突，降低适应度
                } else {
                    courseTimeDict[`${timeslot}-${course.name}`] = room; // 记录课程安排
                    fitnessScore += course.priority; // 加上课程优先级
                    teacherAssignedCount[course.teacher]++; // 增加教师安排计数

                    // 检测学生时间冲突
                    for (let student of this.studentList) {
                        if (student.enrolledCourses.includes(course)) {
                            if (student.timeSlots.includes(timeslot)) {
                                fitnessScore -= 3; // 冲突，降低适应度
                                this.conflictInfoList.push({ student: student.name, course: course.name, timeslot }); // 记录冲突信息
                            } else {
                                student.timeSlots.push(timeslot); // 更新学生的上课时间
                            }
                        }
                    }
                }

                // 检查教师不可上课时间
                if (course.unavailableTimeslots.includes(timeslot)) {
                    fitnessScore -= 3; // 降低适应度
                }
            }
        }

        // 评估教师安排的均衡性
        for (let [teacher, count] of Object.entries(teacherAssignedCount)) {
            if (count === 0) {
                fitnessScore -= 5; // 教师未被分配
            } else if (count >= 3) {
                fitnessScore -= 2; // 教师安排过多
            } else {
                fitnessScore += 1; // 教师安排均衡
            }
```

```
        }

        return fitnessScore; // 返回适应度分数
    }

    // 基因变异操作
    mutate() {
        const keys = Object.keys(this.scheduleDict); // 获取所有安排的键
        const randomKey = keys[Math.floor(Math.random() * keys.length)]; // 随机选择一个安排
        const randomCourse = this.courseList[Math.floor(Math.random() * this.courseList.length)]; // 随机选择一门课程
        this.scheduleDict[randomKey] = randomCourse; // 替换该位置的课程
    }

    // 基因交叉操作
    crossover(other) {
        const crossoverIndex = Math.floor(Math.random() * Object.keys(this.scheduleDict).length); // 随机选择交叉基因的起点
        const child1Schedule = new Schedule(this.courseList, this.timeslotsList, this.roomList, this.studentList);
        const child2Schedule = new Schedule(this.courseList, this.timeslotsList, this.roomList, this.studentList);
        let index = 0;

        for (let key in this.scheduleDict) {
            if (index <= crossoverIndex) {
                child1Schedule.scheduleDict[key] = this.scheduleDict[key]; // 子代1继承本个体基因
                child2Schedule.scheduleDict[key] = other.scheduleDict[key]; // 子代2继承其他个体基因
            } else {
                child1Schedule.scheduleDict[key] = other.scheduleDict[key]; // 子代1继承其他个体基因
                child2Schedule.scheduleDict[key] = this.scheduleDict[key]; // 子代2继承本个体基因
            }
            index++;
        }

        return [child1Schedule, child2Schedule]; // 返回两个新子代
    }

    // 显示排课结果
    display() {
        let unassignedTeachers = new Set(this.courseList.map(course => course.teacher)); // 未安排教师
        let unassignedRooms = new Set(this.roomList); // 未安排教室
        let unassignedStudents = new Set(this.studentList.map(student => student.name)); // 未安排学生

        for (let key in this.scheduleDict) {
            let [timeslot, room] = key.split('-');
            let course = this.scheduleDict[key];

            if (course) {
```

```
                let enrolledStudents = this.studentList.filter(student => student.enrolledCourses.includes(course)).map(student => student.name); // 选课学生列表
                let enrolledStudentStr = enrolledStudents.length > 0 ? enrolledStudents.join(', ') : "No student enrolled"; // 选课学生字符串
                let teacherUnavailable = course.unavailableTimeslots.includes(timeslot) ? " | Teacher unavailable during this time" : ""; // 教师不可用提示

                // 初始化冲突信息
                let conflictStr = this.conflictInfoList.filter(info => info.course === course.name && info.timeslot === timeslot)
                    .map(info => ` | Conflict: ${info.student}`).join('');

                console.log(`${timeslot} in ${room}: ${course.name} by ${course.teacher} (Priority: ${course.priority}) | Enrolled: ${enrolledStudentStr}${conflictStr}${teacherUnavailable}`);

                // 从未安排列表中移除教师和教室
                unassignedTeachers.delete(course.teacher);
                unassignedRooms.delete(room);
            } else {
                console.log(`${timeslot} in ${room}: Free`);
            }
        }

        // 打印未安排的教师、教室和学生
        console.log("Unassigned Teachers: " + (unassignedTeachers.size > 0 ? Array.from(unassignedTeachers).join(', ') : "All teachers assigned."));
        console.log("Unassigned Rooms: " + (unassignedRooms.size > 0 ? Array.from(unassignedRooms).join(', ') : "All rooms assigned."));
        console.log("Unassigned Students: " + (unassignedStudents.size > 0 ? Array.from(unassignedStudents).join(', ') : "All students enrolled."));
    }
}

// 初始化种群
function initializePopulation(courseList, timeslotsList, roomList, studentList, populationSize = 100) {
    let population = []; // 种群列表初始化
    for (let i = 0; i < populationSize; i++) {
        let schedule = new Schedule(courseList, timeslotsList, roomList, studentList); // 实例化一个个体
        for (let key in schedule.scheduleDict) {
            schedule.scheduleDict[key] = courseList[Math.floor(Math.random() * courseList.length)]; // 随机初始化课程安排
        }
        population.push(schedule); // 加入种群列表
    }
    return population; // 返回初始化的种群列表
}

// 交叉和变异获得下一代种群
function geneticEvolution(population, retentionNumber) {
    population.sort((a, b) => b.fitness() - a.fitness()); // 按适应度排序

    let nextGeneration = []; // 下一代个体列表
```

```
    nextGeneration.push(...population.slice(0, retentionNumber)); // 保留最优个体

    // 选择和交叉，直到达到种群大小
    while (nextGeneration.length < population.length) {
        let parent1 = population[Math.floor(Math.random() * 10)]; // 随机选择父母1
        let parent2 = population[Math.floor(Math.random() * 10)]; // 随机选择父母2
        let [child1, child2] = parent1.crossover(parent2); // 交叉生成两个子代
        nextGeneration.push(child1, child2); // 加入下一代个体列表
    }

    // 子代变异
    for (let child of nextGeneration) {
        if (Math.random() < 0.3) {
            child.mutate(); // 随机变异
        }
    }

    return nextGeneration; // 返回下一代
}

// 主程序入口
const courseList = [
    new Course("class1", 'teacher1', 1, 1, ["time2", "time4"]),
    new Course("class2", "teacher2", 1, 1, ["time1"]),
    new Course("class3", "teacher3", 1, 1),
    new Course("class4", "teacher4", 1, 1, ["time3"]),
    new Course("class5", "teacher5", 1, 1),
    new Course("class6", "teacher6", 1, 1, ["time5", "time6"]),
    new Course("class7", "teacher7", 1, 1),
    new Course("class8", "teacher8", 1, 1, ["time2"]),
    new Course("class9", "teacher9", 1, 1),
    new Course("class10", "teacher10", 1, 1, ["time1", "time6"]),
    new Course("class11", 'teacher11', 1, 1),
    new Course("class12", "teacher12", 1, 1, ["time1"]),
    new Course("class13", "teacher13", 1, 1),
    new Course("class14", "teacher14", 1, 1, ["time3"]),
    new Course("class15", "teacher15", 1, 1),
    new Course("class16", "teacher16", 1, 1, ["time5"]),
    new Course("class17", "teacher17", 1, 2),
    new Course("class18", "teacher18", 1, 2, ["time2"]),
    new Course("class19", "teacher19", 1, 3),
    new Course("class20", "teacher20", 1, 3, ["time4"]),
];

// 实例时间段和教室
const timeslotsList = ["time1", "time2", "time3", "time4", "time5", "time6"];
const roomList = ["Room1", "Room2", "Room3"];

// 示例学生
const studentList = [
    new Student("student1"), new Student("student2"), new Student("student3"),
```

```
    new Student("student4"), new Student("student5"), new Student("student6"),
    new Student("student7"), new Student("student8"), new Student("student9"),
    new Student("student10"),
];

// 学生选课
studentList[0].enroll(courseList[0]);
studentList[0].enroll(courseList[1]);
studentList[1].enroll(courseList[2]);
studentList[1].enroll(courseList[3]);
studentList[2].enroll(courseList[4]);
studentList[2].enroll(courseList[12]);
studentList[2].enroll(courseList[13]);
studentList[2].enroll(courseList[14]);
studentList[3].enroll(courseList[5]);
studentList[3].enroll(courseList[15]);
studentList[3].enroll(courseList[16]);
studentList[4].enroll(courseList[6]);
studentList[4].enroll(courseList[17]);
studentList[4].enroll(courseList[18]);
studentList[4].enroll(courseList[19]);
studentList[5].enroll(courseList[7]);
studentList[6].enroll(courseList[8]);
studentList[7].enroll(courseList[9]);
studentList[8].enroll(courseList[10]);
studentList[9].enroll(courseList[11]);

// 序号2学生多选课程
studentList[2].enroll(courseList[15]);
studentList[2].enroll(courseList[16]);
studentList[2].enroll(courseList[17]);
studentList[2].enroll(courseList[18]);

// 种群大小和适应度阈值
const populationSize = 100; // 种群大小
const fitnessThreshold = 50; // 适应度阈值
const retentionNumber = 5; // 保留最优的个体数量

// 获得初始化的种群
let population = initializePopulation(courseList, timeslotsList, roomList, studentList, populationSize);

// 初始化进化次数
let generation = 0;
while (true) {
    // 交叉和变异获得下一代种群
    population = geneticEvolution(population, retentionNumber);

    // 获取最优个体
    let bestSchedule = population.reduce((best, current) => current.fitness() > best.fitness() ? current : best); // 找到适应度最高的个体
    let bestFitness = bestSchedule.fitness();

    // 打印当前代的最优适应度
    generation++;
```

```
    console.log(`Generation ${generation}: Best Fitness = ${bestFitness}`);
    bestSchedule.display();
    console.log("-".repeat(40));

    // 如果达到了适应度要求，退出循环
    if (bestFitness >= fitnessThreshold) {
        break;
    }
}

console.log("Final Best Schedule:");
bestSchedule.display();

```